

Strategic Framework for Predictive Talent Mapping and Event Optimization using Aadhaar Transactional Analytics

January 18, 2026

1 Strategic Framework for Predictive Talent Mapping and Event Optimization using Aadhaar Transactional Analytics

1.0.1 Author Details

Name: Kishor S

Date: January 18, 2026

Email: kishorjksk2006@gmail.com

Website: kshrs.github.io

1.1 Problem Statement:

National upskilling initiatives and hackathons often lack data-driven insights for optimal planning and location selection. Traditional approaches fail to accurately identify where youth populations are most active and which sectors align with regional industrial strengths. This results in inefficient resource allocation and missed opportunities to connect upskilling programs with local industry needs and startup ecosystems.

1.2 Approach:

This project develops a data-driven framework that combines multiple datasets to optimize hackathon planning:

Demand-Side Analysis:

- Leverages high-frequency Aadhaar transactional data
- Identifies the ``Active Youth Pipeline'' by tracking spikes in:
 - Mandatory Biometric Updates (MBU) at age 15
 - Demographic Corrections at age 17
- Provides granular, district-level youth activity insights

Supply-Side Integration:

- AISHE College/University Dataset
- DPIIT Recognized Startup Data
- Major Industry Information (district level)

Output:

- Identifies sectoral trends at the district level
- Aligns State/District objectives with industry presence
- Enables targeted youth upskilling through industry-personal connections
- Provides smarter, data-backed solutions for hackathon planning and resource deployment

1.3 What this Project Solves:

1. Given a government initiative/hackathon, what is the best location to conduct it across India.
2. Identify the active Startup and Industry trend to align with Government initiative of upskilling the youth.
3. Analyze data regarding startup trend, industries and college distribution across India.

1.4 Why this Solution and Problem statement:

When a government event or initiative of conducting a hackathon takes place, a **Student** faces the following struggles,

1. The event/competition is held at a place far away from most participants which makes the ability of the student to participate offline harder, due to lack of preparedness, ticket issues, etc..
2. The event doesn't incorporate industry professionals and startup personals to inspire the participants and know more about that field.
3. The officials face the struggle of choosing an appropriate location for conducting the event which enables them to connect with industries and startup professionals.

1.5 Dataset Info:

- **Aadhaar Enrolment Datset:** [Dataset Link](#) (Open Government Data(OGD Platform))
- **Aadhaar Demographic Update Datset:** [Dataset Link](#) (Open Government Data(OGD Platform))
- **Aadhaar Biometric Update Datset:** [Dataset Link](#) (Open Government Data(OGD Platform))
- **AISHE College/University Survey 2026:** [Dataset Link](#) (aishe.gov.in)
- **DPIIT Recognized Startup Dataset Feb 2025:** [Dataset Link](#) (dataful.in)
- **District-wise Industry Info Dataset Mar 2024:** [Dataset Link](#) (Open Government Data(OGD Platform))
- **Topo Map of India and States for Visualiztion:** [Dataset Link](#) (github.com/udit-001)

Major Column Data Used: Date, Year, State, District, Pincode, Age_0_5, Age_5_17, Age_17, Age_18, College_Name, Discipline Group, Industry, Startups_Recognized, etc..

1.6 Major Mathematical Framework & Administrative Assumptions:

1.6.1 Core Assumptions:

- **The Age 15 MBU:** Mandatory Biometric Updates (MBU) are legally required at age 15. We assume spikes in the Age_5_17 biometric dataset (minus new enrolments) represent the core 15-year-old ``Youth Pipeline."

- **Board Exam Alignment:** Demographic updates (name/address/DoB) peaking at **Age_17** are assumed to be triggered by students aligning Aadhaar details with Class 10/12 certificates for college admissions.
 - **The Formalization Gap:** New enrolments at **Age_18** are assumed to represent youth entering the formal economy (opening bank accounts or applying for scholarships) for the first time.
 - **High Saturation Proxy:** With Jan 2026 Aadhaar saturation at **~95%**, we assume transactional density is a high-fidelity proxy for actual population distribution and activity.
-

1.6.2 The Youth Population Index (*YPI*)

The *YPI* quantifies the density of ``Active'' youth in a district. It measures real-world engagement and intent rather than just a static headcount.

$$YPI = \frac{(B_{5-17} - E_{5-17}) + (1.2 \times D_{17}) + E_{18}}{P_{saturated}} \times 100$$

Breakdown:

- $(B_{5-17} - E_{5-17})$: Isolates the mandatory update spike to identify the Age 15 pipeline.
 - $1.2 \times D_{17}$: Demographic updates, weighted higher (1.2) due to their strong correlation with migration and higher education intent.
 - E_{18} : New adult entrants, signaling digital and financial inclusion.
 - $P_{saturated}$: Normalization factor based on Jan 2026 district saturation benchmarks.
-

1.6.3 The Venue Success Score (*VSS*)

The *VSS* is the final decision metric used to rank districts for a specific hackathon or upskilling theme.

$$VSS = (\omega_1 \times YPI) + (\omega_2 \times T_{density}) + (\omega_3 \times I_{base}) + (\omega_4 \times S_{hub})$$

Explanation of Weightage:

- $\omega_1 \cdot YPI$ (**30%**): Ensures a high volume of active, ``identity-ready'' youth.
- $\omega_2 \cdot T_{density}$ (**30%**): Matches district talent (AISHE college data) with the specific theme.
- $\omega_3 \cdot I_{base}$ (**20%**): Confirms availability of local industrial partners for mentorship and placement.
- $\omega_4 \cdot S_{hub}$ (**20%**): Validates the existing startup culture and innovation sentiment in that sector.

2 Project Code:

Note: Make sure to run all the notebook cells iteratively to avoid any errors and install all the necessary python libraries

2.1 Imported Libraries:

```
[31]: # Data Handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
import fiona

import math
```

2.2 Setup of datasets:

2.2.1 Aadhaar Enrolment, Demographic and Biometric Update Datasets:

```
[32]: enrolment_files = [
    "./assets/api_data_aadhar_enrolment/api_data_aadhar_enrolment_0_500000.csv",
    "./assets/api_data_aadhar_enrolment/
    ↪api_data_aadhar_enrolment_500000_1000000.csv",
    "./assets/api_data_aadhar_enrolment/
    ↪api_data_aadhar_enrolment_1000000_1006029.csv",
]
enrolment_df = pd.concat(map(pd.read_csv, enrolment_files), ignore_index=True)
## Standard Column Naming
enrolment_df.columns = ["Date", "State", "District", "Pincode", "Age_0_5",
    ↪"Age_5_17", "Age_18"]
enrolment_df.head()
```

```
[32]:
```

	Date	State	District	Pincode	Age_0_5	Age_5_17	\
0	02-03-2025	Meghalaya	East Khasi Hills	793121	11	61	
1	09-03-2025	Karnataka	Bengaluru Urban	560043	14	33	
2	09-03-2025	Uttar Pradesh	Kanpur Nagar	208001	29	82	
3	09-03-2025	Uttar Pradesh	Aligarh	202133	62	29	
4	09-03-2025	Karnataka	Bengaluru Urban	560016	14	16	

	Age_18
0	37
1	39
2	12
3	15
4	21

```
[33]: demographic_files = [
    "./assets/api_data_aadhar_demographic/api_data_aadhar_demographic_0_500000.
    ↪csv",
```

```

    "./assets/api_data_aadhar_demographic/
↳api_data_aadhar_demographic_500000_1000000.csv",
    "./assets/api_data_aadhar_demographic/
↳api_data_aadhar_demographic_1000000_1500000.csv",
    "./assets/api_data_aadhar_demographic/
↳api_data_aadhar_demographic_1500000_2000000.csv",
    "./assets/api_data_aadhar_demographic/
↳api_data_aadhar_demographic_2000000_2071700.csv",
]

demographic_df = pd.concat(map(pd.read_csv, demographic_files),
↳ignore_index=True)
## Standard Column Naming
demographic_df.columns = ["Date", "State", "District", "Pincode", "Age_5_17",
↳"Age_17"]
demographic_df.head()

```

```

[33]:

```

	Date	State	District	Pincode	Age_5_17	Age_17
0	01-03-2025	Uttar Pradesh	Gorakhpur	273213	49	529
1	01-03-2025	Andhra Pradesh	Chittoor	517132	22	375
2	01-03-2025	Gujarat	Rajkot	360006	65	765
3	01-03-2025	Andhra Pradesh	Srikakulam	532484	24	314
4	01-03-2025	Rajasthan	Udaipur	313801	45	785

```

[34]: biometric_files = [
    "./assets/api_data_aadhar_biometric/api_data_aadhar_biometric_0_500000.csv",
    "./assets/api_data_aadhar_biometric/
↳api_data_aadhar_biometric_500000_1000000.csv",
    "./assets/api_data_aadhar_biometric/
↳api_data_aadhar_biometric_1000000_1500000.csv",
    "./assets/api_data_aadhar_biometric/
↳api_data_aadhar_biometric_1500000_1861108.csv",
]

biometric_df = pd.concat(map(pd.read_csv, biometric_files), ignore_index=True)
## Standard Column Naming
biometric_df.columns = ["Date", "State", "District", "Pincode", "Age_5_17",
↳"Age_17"]
biometric_df.head()

```

```

[34]:

```

	Date	State	District	Pincode	Age_5_17	Age_17
0	01-03-2025	Haryana	Mahendragarh	123029	280	577
1	01-03-2025	Bihar	Madhepura	852121	144	369
2	01-03-2025	Jammu and Kashmir	Punch	185101	643	1091
3	01-03-2025	Bihar	Bhojpur	802158	256	980
4	01-03-2025	Tamil Nadu	Madurai	625514	271	815

2.2.2 AISHE Dataset:

```
[35]: aishe_df = pd.read_excel("./assets/College-ALL COLLEGE.xlsx")
## Standard Column Naming
aishe_df.columns = ["AISHE_CODE", "College_Name", "State", "District",
    ↪ "Website", "Year_Of_Establishment", "Location", "College_Type",
    ↪ "Management", "U-AISHE_CODE", "University_Name", "University_Type"]
aishe_df.drop(aishe_df.index[[0, 1]], inplace=True) # row 0 and 1 are metadata
aishe_df = aishe_df.reset_index(drop=True) # reset index

# Addition of new columns *College_Category*
def categorize_college(name):
    name = str(name).upper()
    if any(word in name for word in ["ENGINEERING", "TECHNOLOGY",
    ↪ "POLYTECHNIC", "I.I.T", "N.I.T"]):
        return "Engineering_Tech"
    elif any(word in name for word in ["MEDICAL", "HEALTH", "PHARMACY",
    ↪ "DENTAL", "NURSING", "PSYCHOLOGY"]):
        return "Health_Science"
    elif any(word in name for word in ["MANAGEMENT", "BUSINESS", "COMMERCE",
    ↪ "MBA", "BBA", "B.COM"]):
        return "Business_Arts"
    elif any(word in name for word in ["AGRICULTURE", "VETERINARY",
    ↪ "FISHERIES"]):
        return "Agri_Science"
    return "General"

# Process AISHE data to include the new value
aishe_df['College_Category'] = aishe_df['College_Name'].
    ↪ apply(categorize_college)

### Uncomment this print the rae of counts of various College_Category
# aishe_df['College_Category'].value_counts()

aishe_df.head()
```

```
[35]: AISHE_CODE      College_Name \
0    C-59634  061-PT. SRI JAGANNATH PRASAD MAHAVIDHYALAYA,DA...
1    C-8888    100001-KCES'S MOOLJEE JAITHA COLLEGE, JALGAON.
2    C-8913  100002-J.D.M.V.P. SAMAJ'S SHRI S. S. PATIL ART...
3    C-8979  100003-EU'S DR. ANNASAHAB G.D.BENDALE MAHILA M...
4    C-8922  100004-Khandesh College Education Society'S Co...

      State District      Website Year_Of_Establishment \
0  Uttar Pradesh  Agra      WWW.JPCOLLEGE.IN.COM      2016
1  Maharashtra  Jalgaon    www.mjcollege.kces.in      1945
2  Maharashtra  Jalgaon  http://www.jdmvpascjal.ac.in      1972
```

3	Maharashtra	Jalgaon	www.agdbmmjal.ac.in	1984
4	Maharashtra	Jalgaon	www.coe.kces.in	1965

	Location	College_Type	Management	U-AISHE_CODE	\
0	Rural	Affiliated College	Private Un-Aided	U-0509	
1	Urban	Affiliated College	Private Aided (Government Aided)	U-0320	
2	Urban	Affiliated College	Private Aided (Government Aided)	U-0320	
3	Urban	Affiliated College	Private Aided (Government Aided)	U-0320	
4	Urban	Affiliated College	Private Aided (Government Aided)	U-0320	

	University_Name	University_Type	\
0	Dr. B. R. Ambedkar University, Agra	State Public University	
1	Kavayitri Bahinabai Chaudhari North Maharashtra...	State Public University	
2	Kavayitri Bahinabai Chaudhari North Maharashtra...	State Public University	
3	Kavayitri Bahinabai Chaudhari North Maharashtra...	State Public University	
4	Kavayitri Bahinabai Chaudhari North Maharashtra...	State Public University	

	College_Category
0	General
1	General
2	Business_Arts
3	General
4	General

2.2.3 DPIIT Recognized Startups Dataset:

```
[36]: startup_df = pd.read_csv("./assets/
↳startups-data-year-state-and-industry-wise-number-of-startups-recognized-by-the-dpiit-as-of
↳csv")
startup_df.drop(['unit', 'note'], axis=1, inplace=True) # these columns are
↳unnecessary
## Standard Column Naming
startup_df.columns = ["Year", "State", "Industry", "Startups_Recognized"]
startup_df.head()
```

[36]:	Year	State	Industry	Startups_Recognized
0	2025	Assam	Analytics	4
1	2025	Assam	Advertising	5
2	2025	Assam	Architecture Interior Design	2
3	2025	Assam	AR VR (Augmented + Virtual Reality)	1
4	2025	Assam	Automotive	7

2.2.4 Major Industries Dataset:

```
[37]: industry_df = pd.read_csv("./assets/  
    ↳District_wise_No_of_Major_Industries_as_on_Mar_2024_0.csv")  
industry_df.head()
```

```
[37]: Sl.No.      District  Sugar  Steel  Automobile  Garments  Casting  \  
0      1.0  Bangalore Urban    1.0   34.0      120.0    886.0    85.0  
1      2.0  Bangalore Rural    0.0    9.0       16.0    56.0    15.0  
2      3.0    Ramanagara    2.0    3.0       17.0    22.0     3.0  
3      4.0    Chitradurga    0.0    3.0        2.0     6.0     0.0  
4      5.0    Davanagere    1.0    6.0        0.0    19.0    14.0  
  
      Cement  Dall Mills  Rice Mills  Pharmaceuticals  Petroleum  Distillaries  \  
0      3.0         0.0         1.0             26.0         1.0          16.0  
1      3.0         0.0         1.0             13.0         4.0           5.0  
2      0.0         0.0         0.0              4.0         0.0           4.0  
3      3.0         0.0        18.0              0.0         0.0           0.0  
4      0.0         0.0       94.0              0.0         0.0           1.0  
  
      Other  
0  6981.0  
1  1076.0  
2   508.0  
3   204.0  
4   149.0
```

2.2.5 Geoframe Setup for Visualization:

```
[38]: file_path = "./india-maps-data/topojson/india.json"  
file_path_tn = './india-maps-data/topojson/states/tamilnadu.json'  
layers = fiona.listlayers(file_path)  
district_layer = layers[0]  
state_layer = layers[1]  
### India Topo Map Usage  
gdf = gpd.read_file(file_path, layer=state_layer)  
### Tamil Nadu Topo Map Usage  
state_gdf = gpd.read_file(file_path_tn, layer=district_layer)
```

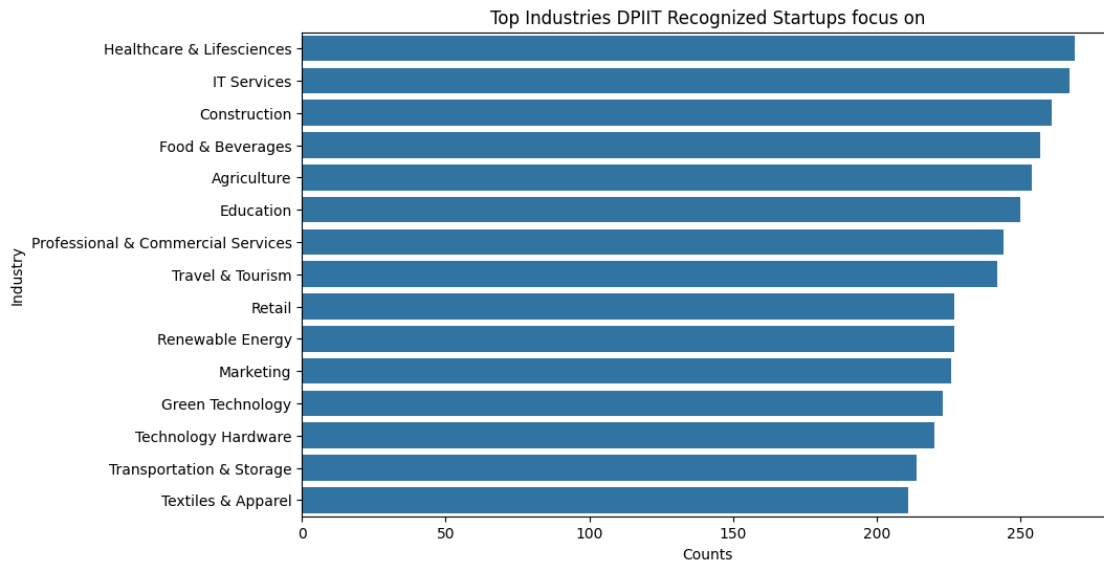
2.3 Raw Data Analysis and Manipulation:

2.3.1 DPIIT Recognized Startups based on Industries:

```
[39]: ### Uncomment this line for raw counts in text  
# print(startup_df['Industry'].value_counts)  
startup_industry_counts = startup_df['Industry'].value_counts().head(15).  
    ↳reset_index()  
startup_industry_counts.columns = ['Industry', 'Counts']
```

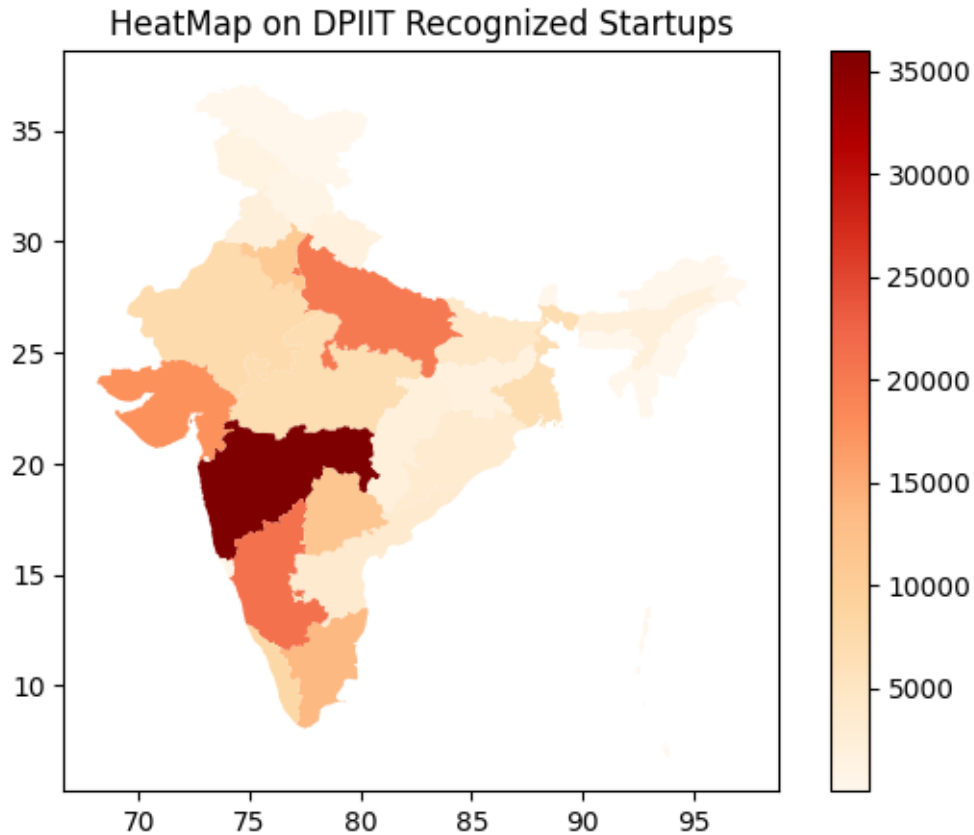


```
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(y = startup_industry_counts['Industry'], x =
    ↳startup_industry_counts['Counts'], ax = ax)
ax.set_title("Top Industries DPIIT Recognized Startups focus on")
plt.show()
```



```
[40]: startup_state_df = startup_df[['State', 'Startups_Recognized']].
    ↳groupby('State', as_index=False).sum()
merged_map = gdf.merge(startup_state_df, left_on="st_nm", right_on="State",
    ↳how="left")

fix, ax = plt.subplots(figsize=(7,5))
merged_map.plot(column="Startups_Recognized", cmap="OrRd", legend=True,
    ↳missing_kwds={"color": "black"}, ax=ax)
plt.title("HeatMap on DPIIT Recognized Startups")
plt.show()
```



```
[41]: macro_industry_df = startup_df.drop(columns=["Year"])
category_map = {
    'AI': 'Tech & AI', 'IT Services': 'Tech & AI', 'Enterprise Software': 'Tech & AI',
    'Analytics': 'Tech & AI', 'Robotics': 'Tech & AI', 'Computer Vision': 'Tech & AI',
    'Healthcare & Lifesciences': 'Healthcare', 'Biotechnology': 'Healthcare',
    'Renewable Energy': 'Sustainability', 'Green Technology': 'Sustainability',
    'Waste Management': 'Sustainability', 'Agriculture': 'Agri-Tech',
    'Automotive': 'Industrial', 'Construction': 'Industrial', 'Technology Hardware': 'Industrial'
}
macro_industry_df['Macro_Sector'] = macro_industry_df['Industry'].
    map(category_map).fillna("Consumer & Services")
state_innovation = macro_industry_df.groupby(['State', "Macro_Sector"]).size().
    unstack(fill_value=0)
state_innovation.head()
```

```

# Filter of Dominant Sectors per State
dominant_sector = macro_industry_df.groupby(['State', 'Macro_Sector']).size().
    ↪reset_index(name='count')
dominant_sector = dominant_sector.sort_values('count', ascending=False).
    ↪drop_duplicates('State')

# Merge with India Shapefile
gdf = gpd.read_file(file_path, layer=state_layer)
merged = gdf.merge(dominant_sector, left_on='st_nm', right_on='State')

# Plotting
fig, axes = plt.subplots(2, 3, figsize=(10, 7))
axes = axes.flatten()

sectors = ['Tech & AI', 'Healthcare', 'Sustainability', 'Industrial',
    ↪'Agri-Tech']

for i, sector in enumerate(sectors):
    # Filter data for that sector
    sector_data = macro_industry_df[macro_industry_df['Macro_Sector'] ==
    ↪sector].groupby('State').size().reset_index(name='count')

    # Merge with map
    temp_map = gdf.merge(sector_data, left_on='st_nm', right_on='State',
    ↪how='left').fillna(0)

    # Plot on specific subplot
    temp_map.plot(column='count', ax=axes[i], cmap='Blues', edgecolor='0.5',
    ↪legend=True)
    axes[i].set_title(f"{sector} Startups", fontsize=15)
    axes[i].axis('off')

plt.tight_layout()
plt.axis('off')
plt.suptitle('Macro Industry Sector Distribution across India for Innovation',
    ↪y=1.02)
plt.show()

#####

n = 16
all_industries = sorted(startup_df['Industry'].unique())[:n]
num_industries = len(all_industries)

cols = 4 # 4 maps per row
rows = math.ceil(num_industries / cols)

```

```

### Uncomment this to loop through all the industries and visualize it
#
# all_industries = sorted(startup_df['Industry'].unique())
# num_industries = len(all_industries)

# cols = 6 # 6 maps per row
# rows = math.ceil(num_industries / cols)
#

# Setup of Figure
# Note: Adjusting figsize to be massive (e.g., 5 inches per map)
fig, axes = plt.subplots(rows, cols, figsize=(cols * 5, rows * 6))
axes = axes.flatten()

# Loop through every industry
for i, industry in enumerate(all_industries):
    # Filter and Aggregate
    industry_data = startup_df[startup_df['Industry'] == industry].
    ↪groupby('State').size().reset_index(name='count')

    # Merge with the India Map
    temp_map = gdf.merge(industry_data, left_on='st_nm', right_on='State',
    ↪how='left').fillna(0)

    # Plotting
    temp_map.plot(column='count', ax=axes[i], cmap='OrRd', edgecolor='0.5',
    ↪linewidth=0.3, legend=True)

    # Formatting
    axes[i].set_title(f"{industry}", fontsize=12, fontweight='bold')
    axes[i].axis('off')

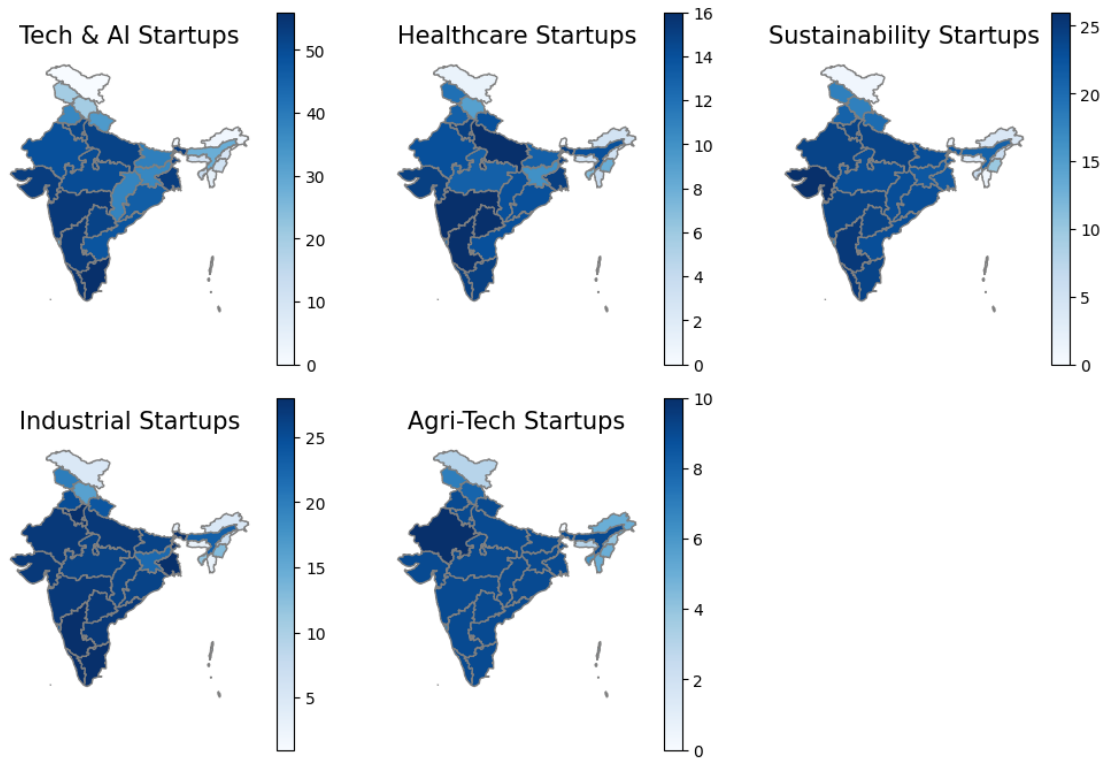
# Hide unused axes
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

# Final Layout
plt.suptitle("The Complete Atlas of Indian Startup Industries (2026)",
    ↪fontsize=40, y=1.02)
plt.tight_layout()

# Save it as a high-res image
# plt.savefig("National_Innovation_Visualization_of_Industries.png", dpi=150,
    ↪bbox_inches='tight')
plt.show()

```

Macro Industry Sector Distribution across India for Innovation



The Complete Atlas of Indian Startup Industries (2026)



2.3.2 Visualization of College Density across India:

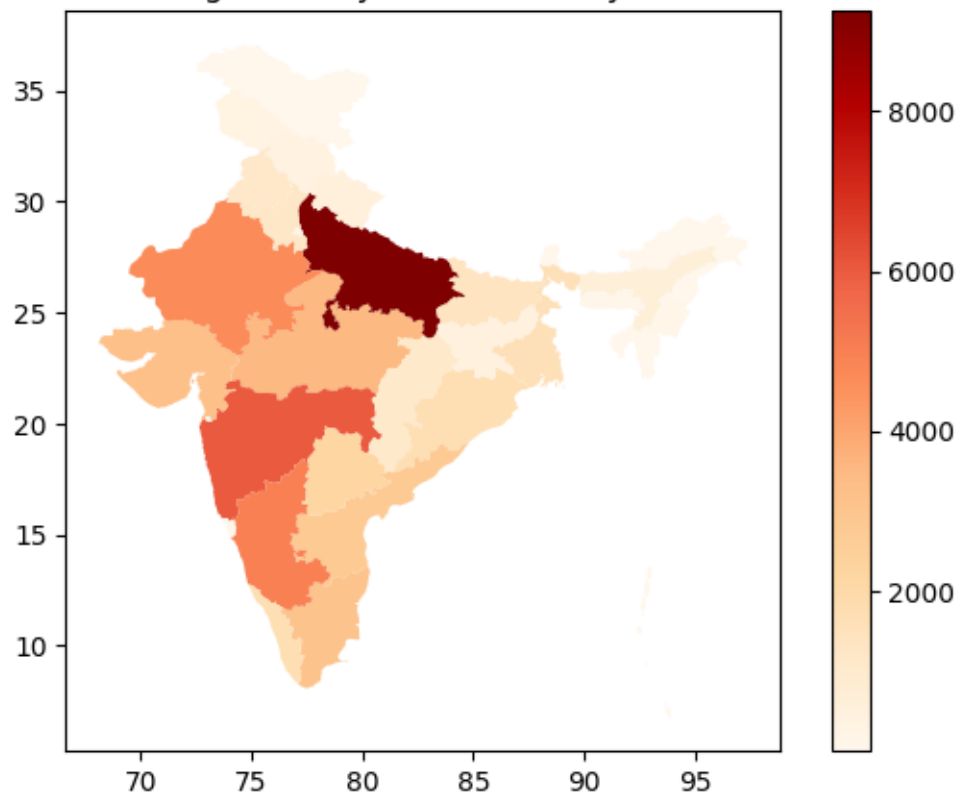
```
[42]: college_list_df = aishe_df.groupby('State', as_index=False).size().
      ↪ rename(columns = {'size': 'Count'})
merged_map = gdf.merge(college_list_df, left_on="st_nm", right_on="State",
      ↪ how="left")

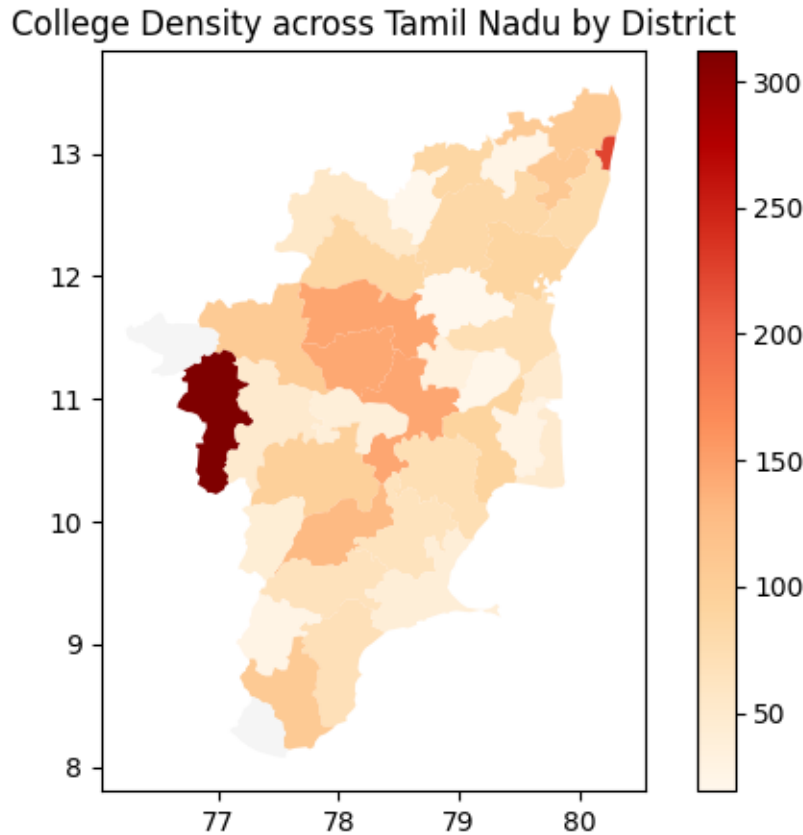
fix, ax = plt.subplots(figsize=(7,5))
merged_map.plot(column="Count", cmap="OrRd", legend=True, missing_kwds={"color":
      ↪ "#F5F5F5"}, ax=ax)
plt.title("College Density across India by State")
plt.show()

college_list_df = aishe_df.groupby('District', as_index=False).size().
      ↪ rename(columns = {'size': 'Count'})
merged_map = state_gdf.merge(college_list_df, left_on="district",
      ↪ right_on="District", how="left")

fix, ax = plt.subplots(figsize=(7,5))
merged_map.plot(column="Count", cmap="OrRd", legend=True, missing_kwds={"color":
      ↪ "#F5F5F5"}, ax=ax)
plt.title("College Density across Tamil Nadu by District")
plt.show()
```

College Density across India by State





2.3.3 Aadhaar Youth Population Index(YPI) Hypothesis

```
[43]: mumbai_enrolment = enrolment_df[enrolment_df["District"] == "Mumbai"]
mumbai_demographic = demographic_df[demographic_df["District"] == "Mumbai"]
mumbai_biometric = biometric_df[biometric_df["District"] == "Mumbai"]

fig, ax = plt.subplots(1, 3, figsize=(15,6))
fig.supxlabel("Age Category")
fig.suptitle("Mumbai: Aadhaar Enrolment, Demographic and Biometric Update_
↳Count")

x = ["Age 0-5", "Age 5-17", "Age >= 18"]
y = [mumbai_enrolment['Age_0_5'].sum(), mumbai_enrolment['Age_5_17'].sum(),
↳mumbai_enrolment['Age_18'].sum()]
sns.barplot(x = x, y = y, ax = ax[0])
ax[0].set_xlabel("Aggregated Count of Aadhaar Enrolments")

x = ["Age 5-17", "Age >= 18"]
y = [mumbai_demographic['Age_5_17'].sum(), mumbai_demographic['Age_17'].sum()]
```

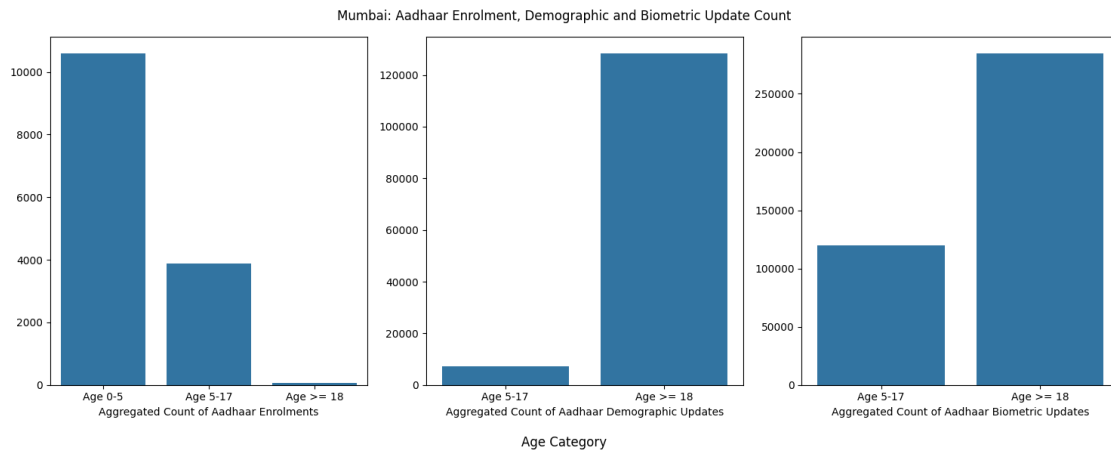
```

sns.barplot(x = x, y = y, ax = ax[1])
ax[1].set_xlabel("Aggregated Count of Aadhaar Demographic Updates")

x = ["Age 5-17", "Age >= 18"]
y = [mumbai_biometric['Age_5_17'].sum(), mumbai_biometric['Age_17'].sum()]
sns.barplot(x = x, y = y, ax = ax[2])
ax[2].set_xlabel("Aggregated Count of Aadhaar Biometric Updates")

plt.tight_layout()
#plt.savefig("mumbai.png")
plt.show()

```



2.3.4 Supportive Inferences from Mumbai Aadhaar Dataset Analysis

Based on the visualized trends in the Mumbai Aadhaar dataset, the following inferences provide a robust foundation for our **Youth Popu Index (YPI)** framework:

- **Dominance of Identity Maintenance:** Demographic and biometric update counts are exponentially higher than new enrolments, confirming that in saturated urban hubs like Mumbai, Aadhaar data is primarily a record of **active lifecycle management** rather than initial registration.
- **The 15-Year MBU Signal:** The **Age 5--17** biometric update count ($\approx 120k$) is nearly **30 times higher** than the enrolment count for the same age group ($\approx 4k$), providing empirical proof that this spike is driven by the **Mandatory Biometric Update (MBU)** requirement at age 15.
- **Adulthood Transition Pivot:** Demographic updates for the **Age ≥ 18** category show massive volume (exceeding 120k), illustrating a **"Transition Pivot"** where youth actively modify their profiles for college admissions, voter registration, or job placements.
- **Saturation Verification:** The near-zero enrolment rate for the **Age ≥ 18** category confirms that Mumbai has reached near-total Aadhaar saturation, meaning nearly all activity in this group represents **established youth** engaging with the digital ecosystem.
- **Concentrated Youth Activity:** The significant volume of Biometric Updates for **Age $\geq$**

18 ($\approx 280k$) suggests that youth in major hubs perform comprehensive ``document hygiene'' to ensure their digital identity is ready for formal professional and educational opportunities.

2.3.5 The Youth Pulse Index (YPI) Formula

Given that the statements above hold true, the following equation does not aim to provide an accurate total population headcount. Instead, it calculates a ``**Pulse Count**'' of individuals who are **actively engaging** with the Aadhaar ecosystem during critical life-stage transitions:

$$YPI = \frac{(B_{5-17} - E_{5-17}) + 1.2 \cdot D_{17} + E_{18}}{100}$$

Note: This index prioritizes ``Active'' youth. While the census counts everyone, this equation identifies the **Market-Ready Youth** who are currently updating documents for education, employment, and formalization.

2.3.6 Calculation of YPI(Youth Population Index):

```
[44]: enrolment_grp = (
        enrolment_df
        .groupby(["State", "District"], as_index=False)
        .sum(numeric_only=True).drop(columns=['Pincode'])
    )
    enrolment_grp.columns = ["State", "District", "E_Age_0_5", "E_Age_5_17", "E_Age_18"]

    demographic_grp = (
        demographic_df
        .groupby(["State", "District"], as_index=False)
        .sum(numeric_only=True).drop(columns=['Pincode'])
    )
    demographic_grp.columns = ["State", "District", "D_Age_5_17", "D_Age_18"]

    biometric_grp = (
        biometric_df
        .groupby(["State", "District"], as_index=False)
        .sum(numeric_only=True).drop(columns=['Pincode'])
    )
    biometric_grp.columns = ["State", "District", "B_Age_5_17", "B_Age_18"]

    YPI = (
        enrolment_grp
        .merge(demographic_grp, on=["State", "District"], how="inner")
        .merge(biometric_grp, on=["State", "District"], how="inner")
    )
    # Calculate Raw Youth Index
```

```

YPI['Youth_Index_Raw'] = ((YPI['B_Age_5_17'] - YPI['E_Age_5_17']) + 1.2 *
    ↪YPI['D_Age_18'] + YPI['E_Age_18']) / 100

# Normalize to 0-1 Range (The "Youth Population Score") [ Min Max Normalization
    ↪]
YPI['Youth_Score'] = (YPI['Youth_Index_Raw'] - YPI['Youth_Index_Raw'].min()) /
    ↪(YPI['Youth_Index_Raw'].max() - YPI['Youth_Index_Raw'].min())

# Reverse Saturation for Population Estimation
# (Using 95% as a general baseline for Jan 2026)
saturation_coeff = 0.95
total_aadhaar_transactions = YPI['E_Age_18'] + YPI['D_Age_18'] +
    ↪YPI['B_Age_18'] # Proxy for adult base

YPI['Est_Total_Pop'] = total_aadhaar_transactions / saturation_coeff
YPI['Projected_Youth_Headcount'] = YPI['Est_Total_Pop'] * 0.28

# Final Sorting
YPI = YPI.sort_values(by='Youth_Score', ascending=False)

### Uncomment this for raw print
#print(YI[['State', 'District', 'Youth_Score', 'Projected_Youth_Headcount']].
    ↪head(10))
YPI[['State', 'District', 'Youth_Score', 'Projected_Youth_Headcount']].head()

```

```

[44]:
      State      District  Youth_Score  Projected_Youth_Headcount
542  Maharashtra      Pune      1.000000      214938.315789
551  Maharashtra      Thane      0.894749      230044.168421
1017 West Bengal  South 24 Parganas  0.712977      146065.389474
258   Gujarat      Surat      0.683247      137292.252632
1004 West Bengal  Murshidabad  0.651040      138437.894737

```

2.4 Analyze and Assign the *Final_Score* for each sector using YPI, Industry, AISHE and Startup datasets:

Current PreAssigned Themes available: **FinTech, HealthTech, Agri_Innovation, Smart_Manufacturing, Fashion_Retail**

```

[45]: college_df = aishe_df.groupby(['State', 'District', 'College_Category']).size().
    ↪unstack(fill_value=0).reset_index()

# Special Case: Normalize District names to handle "Bangalore Urban" vs
    ↪"Bengaluru" inconsistencies
def normalize_names(df, col):
    df[col] = df[col].str.lower().str.replace(' urban', '').str.replace('
    ↪rural', '').str.strip()
    return df

```

```

ypi_df = normalize_names(YPI, 'District')
college_df = normalize_names(college_df, 'District')
industry_df = normalize_names(industry_df, 'District')

# Consolidated Data Frame
master_df = ypi_df.merge(college_df, on=['State', 'District'], how='left') \
               .merge(industry_df, on='District', how='left') \
               .fillna(0)

def hackathon_planner(theme, target_state=None):
    """
    Inputs:
    theme: ['FinTech', 'HealthTech', 'Smart_Manufacturing', 'Agri_Innovation']
    target_state: Filter for specific state (e.g., 'Tamil Nadu')
    """

    # Theme-Specific Weights and Mappings
    # Format: {Theme: (Related_Talent_Column, Related_Industry_Column,
    ↪Related_Startup_Industry)}
    theme_map = {
        'FinTech': ('Business_Arts', 'Other', 'Finance Technology'),
        'HealthTech': ('Health_Science', 'Pharmaceuticals', 'Healthcare &
    ↪Lifesciences'),
        'Smart_Manufacturing': ('Engineering_Tech', 'Steel', 'Robotics'),
        'Agri_Innovation': ('Agri_Science', 'Rice Mills', 'Agriculture'),
        'Fashion_Retail': ('General', 'Garments', 'Fashion')
    }

    if theme not in theme_map:
        return "Theme not found. Please choose from: " + str(list(theme_map.
    ↪keys()))

    tal_col, ind_col, star_ind = theme_map[theme]

    # Calculate Startup Density for that theme per State (from startup_df)
    state_startup_scores = startup_df[startup_df['Industry'] == star_ind].
    ↪groupby('State')['Startups_Recognized'].sum().to_dict()

    # Step-by-Step Scoring
    df = master_df.copy()
    if target_state:
        df = df[df['State'] == target_state]

    # Normalize metrics (Min-Max Scaling) to 0-1
    def scale(series):

```

```

        return (series - series.min()) / (series.max() - series.min() + 1e-9)

    # SUCCESS SCORE FORMULA
    # 30% Youth Activity (Aadhaar) + 30% Local Talent (AISHE) + 20% Industry
    ↪base + 20% Startup Hub
    df['Startup_Score'] = df['State'].map(state_startup_scores).fillna(0)

    df['Final_Score'] = (
        0.30 * scale(df['Youth_Score']) +
        0.30 * scale(df[tal_col]) +
        0.20 * scale(df[ind_col]) +
        0.20 * scale(df['Startup_Score'])
    )

    results = df.sort_values(by='Final_Score', ascending=False).head(5)

    print(f"\n--- TOP 5 LOCATIONS FOR {theme.upper()} HACKATHON ---")
    return results[['State', 'District', 'Youth_Score', tal_col, ind_col,
    ↪'Final_Score']]

```

3 Result:

- What are the **Best Places** to conduct a **Smart Manufacturing Hackathon** in **Tamil Nadu**? Provided the event held should help participants collaborate with industry, startup professionals and in overall help the students preparation and increase participation.

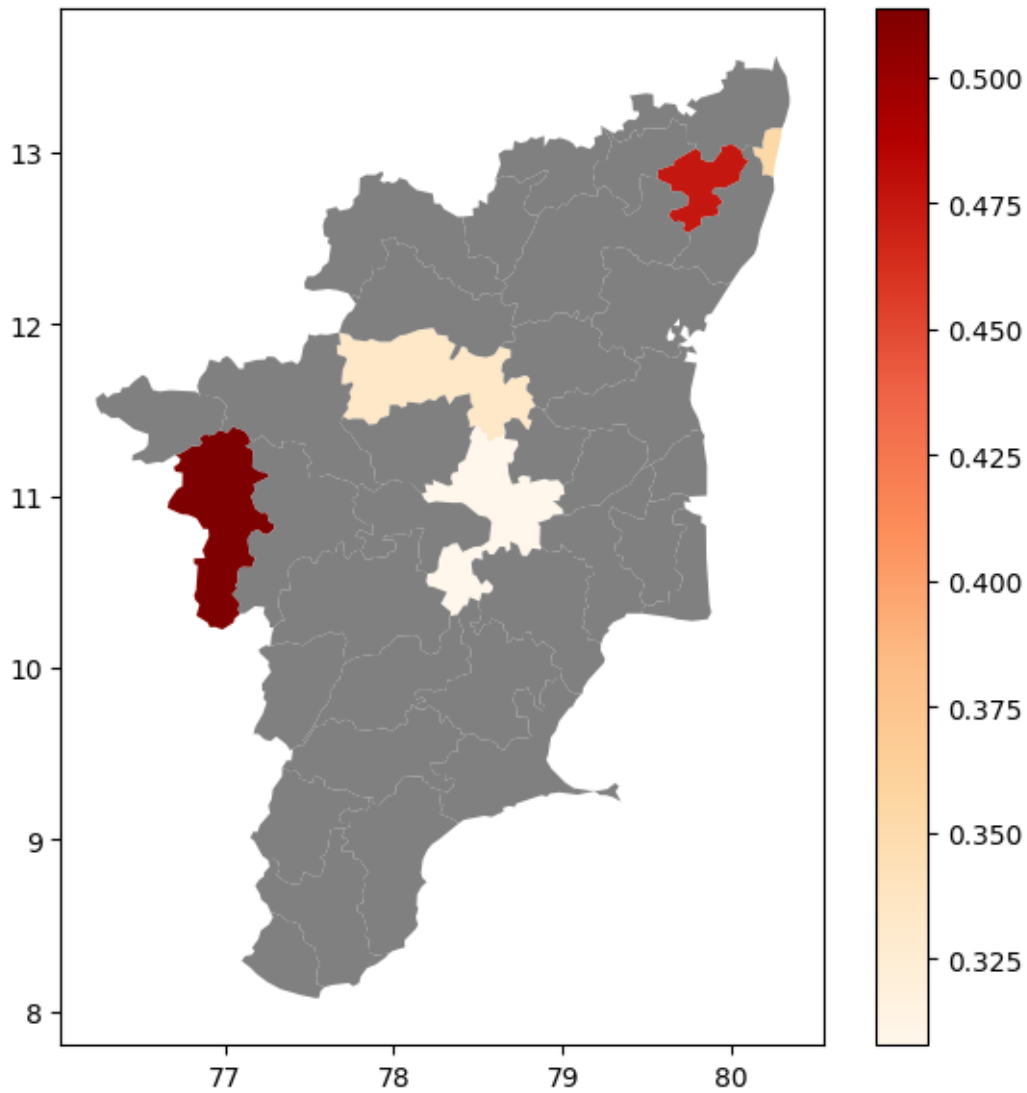
```

[46]: ### Usage Example
# To find the best place for a Smart Manufacturing event in Tamil Nadu:
results = hackathon_planner('Smart_Manufacturing', target_state='Tamil Nadu')
gdf = gpd.read_file("./india-maps-data/topojson/states/tamilnadu.json",
    ↪layer=district_layer)
gdf['district'] = gdf['district'].str.lower()
merged_map = gdf.merge(results, left_on="district", right_on="District",
    ↪how="left")

fig, ax = plt.subplots(figsize=(7,7))
merged_map.plot(column="Final_Score", cmap="OrRd", legend=True,
    ↪missing_kwds={"color": "gray"}, ax=ax)
plt.show()
results

```

--- TOP 5 LOCATIONS FOR SMART_MANUFACTURING HACKATHON ---



```
[46]:      State      District  Youth_Score  Engineering_Tech  Steel  \
129  Tamil Nadu    coimbatore    0.268385         59.0        0.0
70   Tamil Nadu   kancheepuram    0.351355         38.0        0.0
64   Tamil Nadu      chennai    0.356722         13.0        0.0
101  Tamil Nadu      salem      0.303212         18.0        0.0
214  Tamil Nadu  tiruchirappalli    0.198229         30.0        0.0
```

```
      Final_Score
129      0.513617
70      0.475785
64      0.353126
101      0.334083
214      0.307861
```

4 Inference on the Output and Conclusion:

The example demonstrates that **Coimbatore, Tamil Nadu**, is an ideal location for a smart manufacturing hackathon, and for good reason. The city serves as a major hub for:

- Textiles
- Polymers
- Industrial infrastructure
- Motor pumps
- Service and consumer technology sectors

While this methodology for identifying suitable locations is viable, it has limitations in accuracy. This project represents an **innovative approach** by leveraging Aadhaar datasets, including:

- Enrollment records
- Demographic information
- Biometric updates

This enables more informed location decisions.

5 Anomaly Detection and Pattern Recognition

While the primary focus of this framework is predictive talent mapping, the Aadhaar dataset reveals several interesting anomalies that warrant investigation and provide additional insights into youth engagement patterns across India.

5.1 Key Anomalies Detected

1. High Demographic, Low Biometric Updates

Pattern: Districts with disproportionately high demographic updates (Age 17) but low biometric updates (Age 5-17)

Interpretation:

- Indicates migration hotspots where youth are updating addresses for college admissions
- Suggests incoming student population from other districts
- Example districts showing this pattern: Pune, Bengaluru Urban, Hyderabad

```
[47]: # Identify districts where Demographic Updates > 2x Biometric Updates
migration_hotspots = master_df[
    (master_df['D_Age_18'] > 2 * master_df['B_Age_5_17'])
].sort_values(by='D_Age_18', ascending=False)

print("Detected Migration Hotspots (Potential Educational Hubs):")
print(migration_hotspots[['State', 'District', 'D_Age_18', 'B_Age_5_17']].
      ↪head())
```

Detected Migration Hotspots (Potential Educational Hubs):

	State	District	D_Age_18	B_Age_5_17
1	Maharashtra	thane	411163	202896
2	West Bengal	south 24 parganas	373409	102758

4	West Bengal	murshidabad	340843	90308
3	Gujarat	surat	322384	138828
13	West Bengal	north 24 parganas	275457	96368

2. Low Youth Index Despite High College Density

Pattern: Districts with many educational institutions but surprisingly low YPI scores

Interpretation:

- May indicate commuter student populations who maintain Aadhaar records in home districts
- Suggests college infrastructure exists but local youth engagement is lower than expected
- Potential digital literacy gaps or Aadhaar center accessibility issues

```
[48]: # High College Count but Bottom 25% YPI
def normalize_geo(df):
    for col in ['State', 'District']:
        if col in df.columns:
            df[col] = df[col].astype(str).str.lower().str.strip()
            df[col] = df[col].str.replace(' urban', '').str.replace(' rural', '')
    return df

ypi_df = normalize_geo(ypi_df)
college_df = normalize_geo(college_df)
industry_df = normalize_geo(industry_df)
aishe_raw = normalize_geo(aishe_df)

college_counts = aishe_raw.groupby('District').size().
    reset_index(name='College_Count')

master_df = ypi_df.merge(college_df, on=['State', 'District'], how='left') \
    .merge(industry_df, on='District', how='left') \
    .merge(college_counts, on='District', how='left') \
    .fillna(0)

talent_gap_districts = master_df[
    (master_df['College_Count'] > master_df['College_Count'].median()) &
    (master_df['Youth_Score'] < master_df['Youth_Score'].quantile(0.25))
]

print("\nInfrastructure-Engagement Mismatch (Potential Digital Literacy Gaps):")
print(talent_gap_districts[['State', 'District', 'College_Count',
    'Youth_Score']].head())
```

Infrastructure-Engagement Mismatch (Potential Digital Literacy Gaps):

	State	District	College_Count	Youth_Score
778	sikkim	south	37.0	0.018128
793	karnataka	davanagere	99.0	0.017035

794	orissa	bhadrak	55.0	0.017028
796	sikkim	west	38.0	0.016893
804	karnataka	bengaluru south	59.0	0.016448

3. Sudden Enrolment Spikes in Age 18 Category

Pattern: Districts showing abnormally high new enrolments for Age 18 compared to national average

Interpretation:

- Represents formalization frontier - youth entering banking/employment systems for first time
- Often seen in tier-2/tier-3 cities experiencing rapid economic development
- Indicates emerging talent pools not yet captured by traditional university metrics

```
[49]: # Detect districts with Age 18 Enrolment > 3 Standard Deviations from Mean
mean_e18 = master_df['E_Age_18'].mean()
std_e18 = master_df['E_Age_18'].std()

formalization_frontier = master_df[
    master_df['E_Age_18'] > (mean_e18 + 3 * std_e18)
]

print("Formalization Frontier (Rapidly Developing Emerging Pools):")
print(formalization_frontier[['State', 'District', 'E_Age_18']].head())
```

Formalization Frontier (Rapidly Developing Emerging Pools):

	State	District	E_Age_18
8	karnataka	bengaluru	3807
9	karnataka	bengaluru	3807
78	bihar	sitamarhi	2697
79	uttar pradesh	bahraich	2304
549	assam	golaghat	1921

6 Limitations and Future Work

6.1 Current Limitations

6.1.1 Data Constraints

Transactional vs. Saturated Population Data

- The Aadhaar dataset provides transaction counts (enrolments and updates) rather than total saturated population per district
- Limits our ability to calculate true per-capita participation rates, only relative activity indices
- Transaction density serves as a proxy for youth activity, which may be influenced by regional digital literacy variations and administrative drives

Limited Temporal Coverage

- Analysis constrained to a single year (2025-2026) of Aadhaar data

- Cannot identify multi-year trends, seasonal patterns in academic cycles, or long-term migration trajectories
- Temporal snapshot may not represent consistent annual patterns across all districts

District Name Standardization

- Inconsistent naming conventions across datasets required manual normalization (e.g., ``Bangalore Urban" vs. ``Bengaluru")
- Potential for mismatches in district-level aggregations and loss of granular data for newly formed administrative units

6.1.2 Methodological Limitations

Fixed Weight Assignment in VSS Formula

- Venue Success Score weights (30% Youth, 30% Talent, 20% Industry, 20% Startup) are based on logical reasoning rather than empirical validation
- Same weights applied uniformly across all themes, though optimal distributions likely vary by sector
- No ground truth data available to test predictive accuracy

Saturation Coefficient Assumption

- 95% Aadhaar saturation rate applied uniformly across all districts
- Ignores documented regional disparities (urban vs. rural, geographic variations)
- Lacks district-specific saturation benchmarks for precise population estimation

College Categorization Simplification

- Keyword-based matching may misclassify institutions with non-standard naming
- ``General" category serves as catch-all, reducing analytical precision
- Cannot detect interdisciplinary programs spanning multiple categories

6.1.3 Validation Gaps

- Framework not validated against actual participation rates from past government hackathons
- No testing against post-event satisfaction surveys or industry engagement metrics
- Unable to measure prediction error rates or establish confidence intervals without historical outcome data

6.2 Future Work

6.2.1 Machine Learning-Based Optimization

- **Supervised learning models** to learn optimal VSS weight distributions from historical hackathon participation and success metrics
- **Reinforcement learning approach** where the system learns from post-event outcomes (attendance, networking quality, placements) to dynamically adjust weights per theme

6.2.2 Real-Time Population Estimation

- **Integration with Census 2021 projections** to establish district-specific saturation baselines and compute true per-capita youth activity rates

- **Time series forecasting (ARIMA/Prophet)** to predict monthly youth activity trends and identify optimal event timing windows

6.2.3 Geospatial Accessibility Analysis

- **Travel time optimization** incorporating road/rail network connectivity from OpenStreetMap to minimize student travel burden
- **Multi-objective optimization** balancing youth reach and industry partnerships against travel costs and venue capacity constraints

6.2.4 Anomaly Detection & Pattern Recognition

- **Fraud detection** identifying districts with irregular update patterns relative to population benchmarks
- **Migration flow mapping** using pincode-level demographic updates to trace inter-state youth movement and predict emerging talent hubs

6.2.5 Interactive Decision Support System

- **Web-based dashboard** enabling government officials to select themes and receive ranked recommendations with justification
- **Mobile application** for students to receive location-based hackathon notifications and industry mentorship opportunities
- **Feedback loop integration** collecting post-event data to continuously refine model accuracy

6.2.6 Expansion to Other Government Initiatives

- **Skill Development Center placement** using YPI signals to site PM Kaushal Kendra and PMKVY training programs
- **Higher education planning** identifying under-served high-YPI districts for new AISHE institution approvals
- **Startup ecosystem development** recommending incubator locations based on talent pipeline and existing industry clusters

6.2.7 Validation & Impact Assessment

- **Partnership with hackathon organizers** to track actual participation rates vs. predictions and measure student-industry connection quality
- **Open benchmark publication** creating public repository of VSS scores vs. outcomes for transparency and academic research
- **Longitudinal studies** assessing whether the framework reduces geographic inequality in youth opportunity access